TDOA Localization of Fast Radio Bursts

Prepared by
Aric Apoorv Tirkey and Mishthi Sharma

Supervised by
Prof. Mohit Bharadwaj

Introduction:

This project is about localization of Fast Radio Bursts using the method of Time difference of Arrival (TDOA). This method basically uses the difference in the time of arrival of an FRB signal to different telescopes and tries to find the localization area i.e. the area in the space where the FRB might have originated.

We first assume an FRB position , then calculate the time difference. Then with this simulated TDOA we find the localization area by first adding gaussian noise then calculating a Chi square map over the altitude-azimuth grid. Finally we also analyse the change in localization areas with different FRB positions.

This project uses HEALPix that divides the space into equal area pixels via healpy python library to calculate numerical value of Localization Areas.

Objectives:

1. Implement a function altaz_to_unit_vector(alt, az) that generates a unit vector to a given altitude- azimuth and verify correctness by plotting unit vectors for a set of altitudes and azimuths.
2. Choose a true FRB direction . With a baseline length of 15km between two telescopes, compute true geometric delays and add Gaussian timing noise of 10μs.
3. Compute a 2D Chi square map (alt–az grid) and plot the resulting localization band (the 1σ region).
4. Add a third telescope such that the three are not collinear and plot the localization band. Compute the localization area in deg2 (using cos(alt)daltdaz).
5. Compute the 1σ localization areas for a grid of FRB positions over altitudes 20 degrees–85 degrees and azimuths 0 degree–360 degrees.
6. Produce a 2D heatmap of log10(localization area).

Methodology:

1. We define the function named altaz_to_unit_vector that takes altitude and azimuth (int degrees) as input. It first converts these into radians and then finds x,y and z coordinates using x= cos(alt) sin(az),y= cos(alt) cos(az)and z= sin(alt). It stores these in a 1d array and returns the array which represents our unit vector. We used quiver function to plot the position vectors. This function is used to plot arrows between 2 points in 3d coordinate systems.
2. We assumed a true FRB direction and found geometric delays for 2 antenna system using the formula $\Delta ti0 = ti - t0 = (ri - r0)\cdot s/c$, where ri is position vector of ith antenna and s is unit vector of FRB and c is speed of light. We used numpy.random.normal function to generate gaussian noises. This function takes 2 parameters loc/mean and scale/standard deviation and generates a gaussian distribution centred at loc. It returns an array based on specified size.
3. We created alt-az grids using two numpy arrays with their values representing each point in the space. Then we make a 2d array to store Chi square values for all the alt-az points in our grids. We first get the unit vector s for each az-alt point using our function then find time delay and Chi square value using the function given under section 1.5. Then we find the minimum Chi square value using min function. Then we suntract Chi square min from Chi square of each point. Now we use meshgrid with our altitude and azimuth grids and use contour plot to plot the 1 sigma localization patch using the parameter delta Chi square<=2.3.
4. We added one more antenna and used same procedure as before. Now for localization area we sum up dA for all the points inside the localization patch. dA is cos(alt)daltdaz in degree square.
5. We use similar procedure for all the specified FRB points. But instead of using numpy arrays for our alt-az grids we use healpy pixels. For Localization area we sum up dA for all points inside localization patch but here dA is the area of healpy pixels. We store the localization areas in a 2D array according the altitude and azimuth of FRB points.
6. Once Area is found we find log_Area and then contour plot it against FRB altitude and azimuth grids.

Results:

1. We get correct plots for position vectors for three sets of alt-az which confirms our implemented altaz_to_unit_vector works fine.
2. Using the formula gave the time difference in a straightforward way. Using the random.normal function with specified standard deviation resulted in correct gaussian noises.
3. Using alt-az grids as numpy array of points resulted in approximately correct results as we took the gap between two alt and two az to be just 0.1 degrees. Finally we got a ring shaped localization patch for 2 antenna system which was expected.
4. Introducing a third antenna correctly shrinked our localization patch to a compact area. Calculating localization area by summing up dA approximately gave correct areas as our grid spacing is less.

5. We used healpy to speed up our code. It more accurately calculates localization areas as it divides the sky into pixels and no point is lost as in numpy grids.
6. The final 2D heatmap of log(Localization area) shows that localization areas are consistent at higher altitudes and not at all consistent at lower altitudes. The result is accurate and as expected.

Conclusion:

This project successfully finds localization areas for simulated time differences of arrival and can work well with actual time difference. The project is successful in studying the change in localization based on FRB position.

References:

Numpy documentation
Matplotlib documentation
Healpy documentation
https://healpix.sourceforge.io/